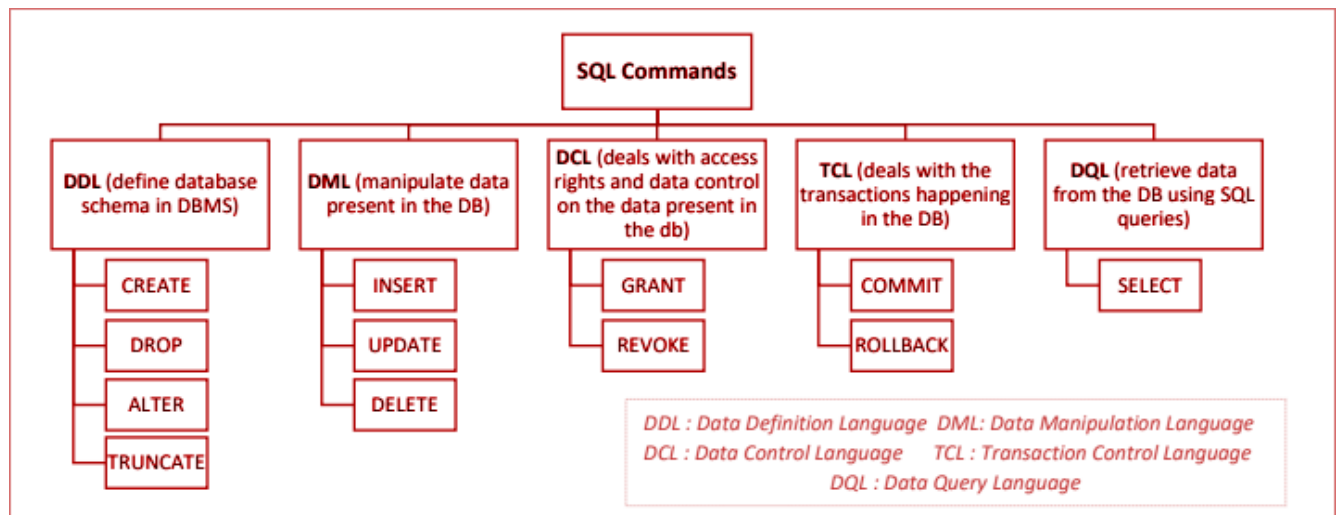


MySQL Statement

- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.



Data Definition Language

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.
- Here are some commands that come under DDL:
-

– CREATE

It is used to create a table & database.

– DROP

It is used to delete both the structure and record stored in the table.

– ALTER

It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute/column or probably to add a new attribute/column.

– TRUNCATE

It is used to delete all the rows from the table and free the space containing the table.

– Rename

It is used to rename the table.

Create database revision;

Use revision;

Create table customers(

Id int ,

Name varchar(20),

Email varchar(25),

City varchar(20)

);

Describe table : desc <table name>

Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.
- Here are some commands that come under DML:
-

– INSERT

- INSERT INTO is used to store data in the tables. The INSERT command creates a new row in the table to store data.
- The data is usually supplied by application programs that run on top of the database.
- Basic syntax:

INSERT INTO 'table_name' (column_1, column_2,...) VALUES (value_1,value_2,...);
ex.

insert into customers values(1, "vipul","vipul@gmail.com","Mumbai");

- The INSERT command can also be used to insert data into a table from another table.

- The basic syntax is as shown below.

INSERT INTO table_1 SELECT * FROM table_2;

– UPDATE

- UPDATE MySQL command is used to modify rows in a table. The update command can be used to update a single field or multiple fields at the same time.
- It can also be used to update a MySQL table with values from another table.
- Syntax

UPDATE 'table_name' SET 'column_name' = 'new_value' [WHERE condition];

– DELETE

- MySQL Delete command is used to delete rows that are no longer required from the database tables.
- It deletes the whole row from the table and returns count of deleted rows.
- Delete command comes in handy to delete temporary or obsolete data from your database.
- The Delete query in MySQL can delete more than one row from a table in a single query.
- This proves to be advantages when removing large numbers of rows from a database table.
- To delete a row in MySQL, the DELETE FROM statement is used:
- Syntax :

DELETE FROM 'table_name' [WHERE condition];

Data Control Language

- DCL commands are used to grant and take back authority from any database user.
- Here are some commands that come under DCL:

– Grant

Grant all privileges <db_name>.<db.table> to 'user'@'localhost' identified by 'password';

Localhost - 127.0.0.1

All - select, insert, update , delete, alter , grant

– Revoke

Revoke all privileges <db_name>.<db.table> from 'user'@'localhost'

Transaction Control Language

- TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.
- These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.
- Here are some commands that come under TCL:

– COMMIT

– ROLLBACK

– SAVEPOINT

- Start transaction

```
mysql> set autocommit = 0
-> ;
Query OK, 0 rows affected (0.00 sec)

mysql> select @@autocommit;
+-----+
| @@autocommit |
+-----+
|             0 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into customers values(1, "vipul","vipul@gmail.com","Mumbai");
Query OK, 1 row affected (0.00 sec)

mysql> insert into customers(name,email,city) values( "Khetesh","khetesh@gmail.com","Mumbai");
Query OK, 1 row affected (0.00 sec)

mysql> select * from customers;
+-----+-----+-----+-----+
| Id    | Name    | Email                | City    |
+-----+-----+-----+-----+
| 1     | vipul   | vipul@gmail.com      | Mumbai  |
| NULL  | Khetesh | khetesh@gmail.com    | Mumbai  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from customers;
Empty set (0.00 sec)
```

```
mysql> set autocommit = 1;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into customers values(1, "vipul","vipul@gmail.com","Mumbai");
Query OK, 1 row affected (0.00 sec)

mysql> insert into customers(name,email,city) values( "Khetesh","khetesh@gmail.com","Mumbai");
Query OK, 1 row affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from customers;
+-----+-----+-----+-----+
| Id    | Name    | Email                | City    |
+-----+-----+-----+-----+
| 1     | vipul   | vipul@gmail.com      | Mumbai  |
| NULL  | Khetesh | khetesh@gmail.com    | Mumbai  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

mysql> set autocommit = 0;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into customers(id,name,email,city) values(3,"Khetesh","khetesh@gmail.com","Mumbai"),(4,"Mohini","Mohini@gmail.com","Pune");
Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> savepoint s1;
Query OK, 0 rows affected (0.00 sec)

mysql> update customers set id=10 where id=4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from customers;

Id	Name	Email	City
1	vipul	vipul@gmail.com	Mumbai
NULL	Khetesh	khetesh@gmail.com	Mumbai
3	Khetesh	khetesh@gmail.com	Mumbai
10	Mohini	Mohini@gmail.com	Pune

4 rows in set (0.00 sec)

mysql> rollback to s1;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from customers;

Id	Name	Email	City
1	vipul	vipul@gmail.com	Mumbai
2	Khetesh	khetesh@gmail.com	Mumbai
3	Khetesh	khetesh@gmail.com	Mumbai
4	Mohini	Mohini@gmail.com	Pune

4 rows in set (0.00 sec)

```
mysql> select * from customers;
```

Id	Name	Email	City
1	vipul	vipul@gmail.com	Mumbai
NULL	Khetesh	khetesh@gmail.com	Mumbai
3	Khetesh	khetesh@gmail.com	Mumbai
4	Mohini	Mohini@gmail.com	Pune

```
4 rows in set (0.00 sec)
```

```
mysql> update customers set id=2 where id is null;
```

```
Query OK, 1 row affected (0.00 sec)
```

```
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> commit;
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> rollback;
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select * from customers;
```

Id	Name	Email	City
1	vipul	vipul@gmail.com	Mumbai
2	Khetesh	khetesh@gmail.com	Mumbai
3	Khetesh	khetesh@gmail.com	Mumbai
4	Mohini	Mohini@gmail.com	Pune

```
4 rows in set (0.00 sec)
```

```
mysql> |
```

Data Query Language

- DQL is used to fetch the data from the database.
- It uses only one command:

– SELECT

Select * from <table_name>;

Select col1, col2 from <table_name>;

Select col1, col2 from <table_name> where {condition}

MySQL Data Types

- A Data Type specifies a particular type of data, like integer, floating points, Boolean, etc.
- It also identifies the possible values for that type, the operations that can be performed on that type, and the way the values of that type are stored.
- In MySQL, each database table has many columns and contains specific data types for each column.
- We can determine the data type in MySQL with the following characteristics:
 - The type of values (fixed or variable) it represents.
 - The storage space it takes is based on whether the values are a fixed-length or variable length.

MySQL Data Types: Numeric

- MySQL has all essential SQL numeric data types. These data types can include the exact numeric data types (For example, integer, decimal, numeric, etc.), as well as the approximate numeric data types (For example, float, real, and double precision).
- It also supports BIT datatype to store bit values. In MySQL, numeric data types are categorized into two types, either signed or unsigned except for bit data type.

TINYINT

- It is a very small integer that can be signed or unsigned.
- If signed, the allowable range is from -128 to 127. If unsigned, the allowable range is from 0 to 255. We can specify a width of up to 4 digits.
- It takes 1 byte for storage.

SMALLINT

- It is a small integer that can be signed or unsigned.
- If signed, the allowable range is from -32768 to 32767. If unsigned, the allowable range is from 0 to 65535.
- We can specify a width of up to 5 digits. It requires 2 bytes for storage.

MEDIUMINT

- It is a medium-sized integer that can be signed or unsigned.
- If signed, the allowable range is from - 8388608 to 8388607. If unsigned, the allowable range is from 0 to 16777215.
- We can specify a width of up to 9 digits. It requires 3 bytes for storage.

INT

- It is a normal-sized integer that can be signed or unsigned.
- If signed, the allowable range is from - 2147483648 to 2147483647. If unsigned, the allowable range is from 0 to 4294967295.
- We can specify a width of up to 11 digits. It requires 4 bytes for storage.

BIGINT

- It is a large integer that can be signed or unsigned.
- If signed, the allowable range is from -9223372036854775808 to 9223372036854775807. If unsigned, the allowable range is from 0 to 18446744073709551615.
- We can specify a width of up to 20 digits. It requires 8 bytes for storage.

FLOAT(m,d)

- It is a floating-point number that cannot be unsigned.
- You can define the display length (m) and the number of decimals (d).

DOUBLE(m,d)

- It is a double-precision floating-point number that cannot be unsigned. You can define the display length (m) and the number of decimals (d).

BIT(m)

- It is used for storing bit values into the table column. Here, M determines the number of bit per value that has a range of 1 to 64.

BOOL

- It is used only for the true and false condition. It considered numeric value 1 as true and 0 as false.

BOOLEAN

- It is Similar to the BOOL.

MySQL Data Types: Date and Time

YEAR[(2|4)]

- Year value as 2 digits or 4 digits. The default is 4 digits. It takes 1 byte for storage.

DATE

- Values range from '1000-01-01' to '9999-12-31'. Displayed as 'yyyy-mm-dd'. It takes 3 bytes for storage.

TIME

- Values range from '-838:59:59' to '838:59:59'. Displayed as 'HH:MM:SS'. It takes 3 bytes plus fractional seconds for storage.

DATETIME

- Values range from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Displayed as 'yyyy-mm-dd hh:mm:ss'.
- It takes 5 bytes plus fractional seconds for storage.

TIMESTAMP(m)

- Values range from '1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' TC. Displayed as 'YYYY-MM-DD HH:MM:SS'.
- It takes 4 bytes plus fractional seconds for storage.

MySQL Data Types: String

- The string data type is used to hold plain text and binary data, for example, files, images, etc.
- MySQL can perform searching and comparison of string value based on the pattern matching such as LIKE operator

CHAR(size)

- It can have a maximum size of 255 characters.
- Here size is the number of characters to store. Fixed-length strings.

Char(4)

“ab__”

“Abcd”

“ABCDE” -> ABCD

VARCHAR(size)

- It can have a maximum size of 255 characters.
- Here size is the number of characters to store. Variable-length string.

varchar(4)

“Ab”

“Abcd”

ABCDE -> ABCDE

ADD Snippets for varchar(4), char(4) with greater values.

TINYTEXT(size)

- It can have a maximum size of 255 characters.

• TEXT(size)

- Maximum size of 65,535 characters.

MEDIUMTEXT(size)

- It can have a maximum size of 16,777,215 characters.

LONGTEXT(size)

- It can have a maximum size of 4GB or 4,294,967,295 characters.

BINARY(size)

- It can have a maximum size of 255 characters.

ENUM

- It takes 1 or 2 bytes that depend on the number of enumeration values. An ENUM can have a maximum of 65,535 values.
- It is short for enumeration, which means that each column may have one of the specified possible values. It uses numeric indexes (1, 2, 3...) to represent string values.

Gender enum("male", "female")

Creating a table

- MySQL allows us to create a table into the database by using the CREATE TABLE command. Following is a generic
- syntax for creating a MySQL table in the database.

CREATE TABLE [IF NOT EXISTS] table_name(

column_definition1,
column_definition2,

.....,
table_constraints
);

EX.

```
CREATE TABLE student(  
id int NOT NULL AUTO_INCREMENT,  
name varchar(45) NOT NULL,  
age int NOT NULL,  
PRIMARY KEY (id)  
);
```

Primary Key

- MySQL primary key is a single or combination of the field, which is used to identify each record in a table uniquely.
- If the column contains primary key constraints, then it cannot be null or empty. A table may have duplicate columns, but it can contain only one primary key.
- It always contains unique value into a column.
- Following are the rules for the primary key:
 - The primary key column value must be unique.
 - Each table can contain only one primary key.
 - The primary key column cannot be null or empty.
 - MySQL does not allow us to insert a new row with the existing primary key.
 - It is recommended to use INT or BIGINT data type for the primary key column.

EX. - 1

```
Mysql> CREATE TABLE Login(  
login_id INT AUTO_INCREMENT PRIMARY KEY,  
username VARCHAR(40),  
password VARCHAR(55),  
email VARCHAR(55)  
);
```

Ex - 2

```
mysql> CREATE TABLE Students (  
Roll_No int,
```

Name varchar(45) NOT NULL,
Age int,
Primary Key(Roll_No));
Foreign Key

- The foreign key is used to link one or more than one table together. It is also known as the referencing key.
- A foreign key matches the primary key field of another table. It means a foreign key field in one table refers to the primary key field of the other table.
- It identifies each row of another table uniquely that maintains the referential integrity in MySQL.
- A foreign key makes it possible to create a parent- child relationship with the tables.
- In this relationship, the parent table holds the initial column values, and column values of child table reference the parent column values.
- MySQL allows us to define a foreign key constraint on the child table.
- MySQL defines the foreign key in two ways:
 - Using CREATE TABLE Statement
 - Using ALTER TABLE Statement

EX

```
mysql> CREATE TABLE Students (  
Roll_No int,  
Name varchar(45) NOT NULL,  
Age int,  
Primary Key(Roll_No) );
```

```
mysql> CREATE TABLE address(  
id int,  
city_Name varchar(45) NOT NULL,  
pincode bigint,  
Roll_no int,  
Primary Key(id)  
CONSTRAINT fk_student FOREIGN KEY (Roll_No) REFERENCES  
Studnets(Roll_No)  
);
```

NOT NULL is a field attribute, and it is used because we don't want this field to be NULL.

If we try to create a record with a NULL value, then MySQL will raise an error.

UNIQUE: Ensures that all values in a column or a set of columns are distinct. No two rows can have the same value in a UNIQUE constrained column.

DEFAULT: Provides a default value for a column if no value is explicitly specified during an INSERT operation.

Check

- The check constraint is an integrity constraint that controls the value in a particular column.
- It ensures the inserted or updated value in a column must be matched with the given condition.
- In other words, it determines whether the value associated with the column is valid or not with the given condition.
- EX :

```
CREATE TABLE vehicle (  
vehicle_no VARCHAR(18) PRIMARY KEY,  
model_name VARCHAR(45),  
cost_price DECIMAL(10,2) NOT NULL CHECK (cost_price >= 0),  
sell_price DECIMAL(10,2) NOT NULL CHECK (sell_price >= 0) );  
EX.
```

Insert into vehicle values (123, "tataNexon", "10L" , "12L"); no error

Insert into vehicle values (123, "tataNexon", -1 , "12L"); error

The field attribute **AUTO_INCREMENT** specifies MySQL to go ahead and add the next available number to the id field.

Select

The SELECT statement in MySQL is used to fetch data from one or more tables. We can retrieve records of all fields or specified fields that match specified criteria using this statement.

Syntax:

```
SELECT field_name1, field_name 2,... field_nameN  
FROM table_name1, table_name2...  
WHERE condition  
GROUP BY field_name(s)  
HAVING condition  
ORDER BY field_name(s)  
OFFSET M LIMIT N;
```

Where clause

- WHERE Clause in MySQL is a keyword used to specify the exact criteria of data or rows that will be affected by the specified SQL statement.
- The WHERE clause can be used with SQL statements like INSERT, UPDATE, SELECT, and DELETE to filter records and perform various operations on the data.
- Syntax

`SELECT * FROM tableName WHERE condition;`

Where clause with AND

The WHERE condition in MySQL when used together with the AND logical operator, is only executed if ALL filter criteria specified are met.

Where clause with OR

The WHERE clause when used together with the OR operator, is only executed if any or the entire specified filter criteria is met.

Where clause with IN

- The WHERE in MySQL clause, when used together with the IN keyword only affects the rows whose values matches the list of values provided in the IN keyword.
- The MySQL IN statement helps to reduce number of OR clauses you may have to use. The following MySQL WHERE IN query gives rows where membership_number is either 1 , 2 or 3
- ex
- `SELECT * FROM students WHERE roll_no IN (1,2,3);`

Where clause with NOT IN

- The WHERE clause when used together with the NOT IN keyword DOES NOT affects the rows whose values matches the list of values provided in the NOT IN keyword.
- ex
- `SELECT * FROM students WHERE roll_no IN (1,2,3);`

Between

- The BETWEEN operator selects values within a given range.
- The values can be numbers, text, or dates.
- The BETWEEN operator is inclusive: begin and end values are included.
- syntax
- `SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2;`

Is NULL

MySQL IS NULL condition is used to check if there is a NULL value in the expression.

It is used with SELECT, INSERT, UPDATE and DELETE statement.

- Syntax:
 - expression IS NULL

IS Not NULL

- Syntax:
 - expression IS NOT NULL

Like

- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.
- There are two wildcards often used in conjunction with the LIKE operator:
 - The percent sign (%) represents zero, one, or multiple characters
 - The underscore sign (_) represents one, single character
- The percent sign and the underscore can also be used in combinations!

Syntax:

SELECT column1, column2, ... FROM table_name WHERE columnN LIKE pattern;

EX:

WHERE CustomerName LIKE 'a%' - Finds any values that start with "a"

WHERE CustomerName LIKE '%a' - Finds any values that end with "a"

WHERE CustomerName LIKE '%or%' - Finds any values that have "or" in any position

WHERE CustomerName LIKE '_r%' - Finds values that have "r" in the second position

WHERE CustomerName LIKE 'a_%' - Finds any values that start with "a" and are at least 2 characters in length

WHERE CustomerName LIKE '__a__%'

WHERE ContactName LIKE 'a%o%a' - Finds any values that start with "a" and ends with "o"

Distinct

- The DISTINCT keyword that allows us to omit duplicates from our results.
- This is achieved by grouping similar values together .

Aggregate Functions

- MySQL's aggregate function is used to perform calculations on multiple values and return the result in a single value like the average of all values, the sum of all values, and maximum & minimum value among certain groups of values.
- We mostly use the aggregate functions with SELECT statements in the data query languages.

count()

- It returns the number of rows, including rows with NULL values in a group.

sum()

- It returns the total summed values (Non-NULL) in a set.

avg()

- It returns the average value of an expression.

Min()

- It returns the minimum (lowest) value in a set.

max()

- It returns the maximum (highest) value in a set.

Order by

- MySQL ORDER BY is used in conjunction with the SELECT query to sort data in an orderly manner.
- The MySQL ORDER BY clause is used to sort the query result sets in either ascending or descending order.

SELECT statement... [WHERE condition | GROUP BY 'field_name(s)' HAVING condition] ORDER BY 'field_name(s)' [ASC | DESC];

Example:

SELECT * FROM students ORDER BY marks DESC;

group by

- The GROUP BY clause is a SQL command that is used to group rows that have the same values.

- The GROUP BY clause is used in the SELECT statement. Optionally it is used in conjunction with aggregate functions to produce summary reports from the database.
- That's what it does, summarizing data from the database.
- The queries that contain the GROUP BY clause are called grouped queries and only return a single row for every grouped item.

Syntax

SELECT statements... GROUP BY column_name1[,column_name2,...];

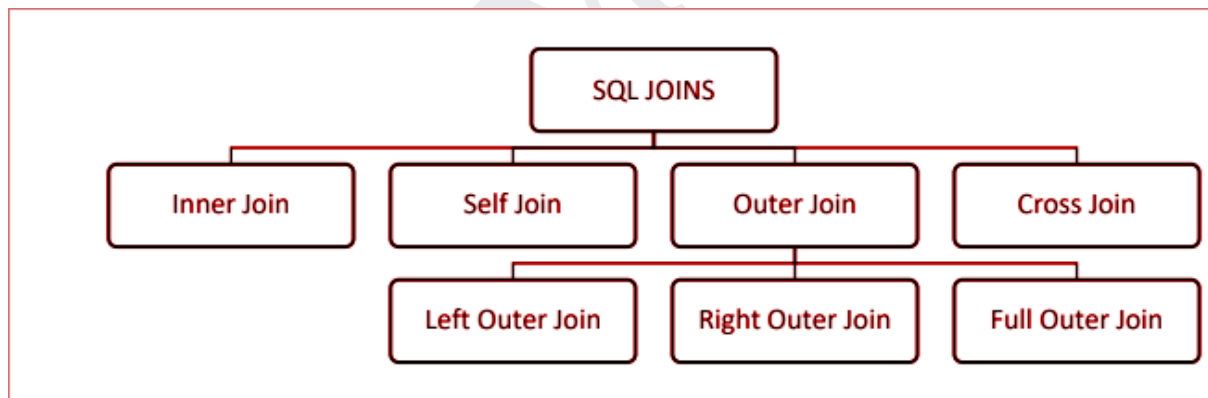
Having Clause

- MySQL HAVING Clause is used with GROUP BY clause. It always returns the rows where condition is TRUE.
- Syntax:
- SELECT expression1, expression2, ... expression_n, aggregate_function(expression) FROM tables [WHERE conditions]

GROUP BY expression1, expression2, ... expression_n

HAVING condition;

Joins



- In a relational database, data is often spread across multiple tables. When retrieving information from multiple tables, we use a database operation called join.
- A JOIN is used to combine rows from two or more tables based on a related column (usually a foreign key–primary key relationship).
- It allows you to query data that is spread across multiple tables.
- Without joins, data would be scattered across multiple tables.
- For example:
 - You have a student's table with student details.

- And a courses table with course details.
 - You need a report showing student name + course name.
- Here we require JOINS

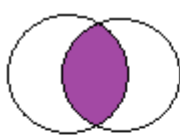
Syntax :

SELECT columns

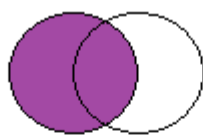
FROM table1

JOIN table2

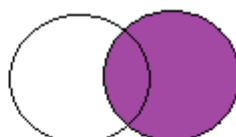
ON table1.common_column = table2.common_column;



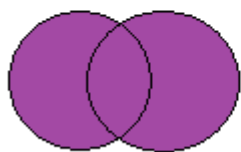
INNER JOIN



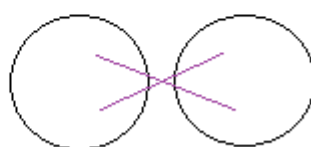
LEFT JOIN



RIGHT JOIN



FULL OUTER JOIN



CROSS JOIN

Inner Join in MySQL

SELECT column_name(s)

FROM table1

INNER JOIN table2

ON table1.column_name = table2.column_name;

Or

SELECT column_name(s)

FROM table1

JOIN table2

ON table1.column_name = table2.column_name;

Left Join in MySQL

SELECT column_name(s)

```
FROM table1  
LEFT JOIN table2  
ON table1.column_name = table2.column_name;
```

Right Join in MySQL

```
SELECT column_name(s)  
FROM table1  
RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

Cross Join in MySQL

```
SELECT column_name(s)  
FROM table1  
CROSS JOIN table2;  
Or
```

```
SELECT column_name(s)  
FROM table1, table2;
```

SELF JOIN

```
SELECT a.column1, b.column2  
FROM table_name AS a  
INNER JOIN table_name AS b  
ON a.id = b.parent_id;
```

UNION A U B

- In MySQL Union, combines results from two or more SELECT queries into a single result set.
- Removes duplicate rows by default (like a mathematical set union).
- Each SELECT must have same number of columns and compatible data types.
- The result is automatically sorted (in many databases, though not guaranteed in MySQL unless ORDER BY is explicitly used).

Syntax:

```
SELECT column_list FROM table1
```

```
UNION
```

```
SELECT column_list FROM table2;
```

UNION ALL

- Also combines multiple SELECT queries,
- Keeps duplicates (does not remove them),
- Faster than UNION because it skips the duplicate-removal step.

Syntax:

```
SELECT column_list FROM table1
```

```
UNION ALL
```

```
SELECT column_list FROM table2;
```

Full Outer Join

- MySQL (up to 8.x) doesn't natively support FULL OUTER JOIN, but we can simulate it easily using UNION.
- A FULL OUTER JOIN returns:
 - All rows from the left table
 - All rows from the right table
 - Matches rows where they exist in both
 - If there's no match, it still returns the row with NULLs for missing columns.
- FULL OUTER JOIN = LEFT JOIN + RIGHT JOIN (combined)
- MySQL has no built-in FULL JOIN, we emulate it using UNION.

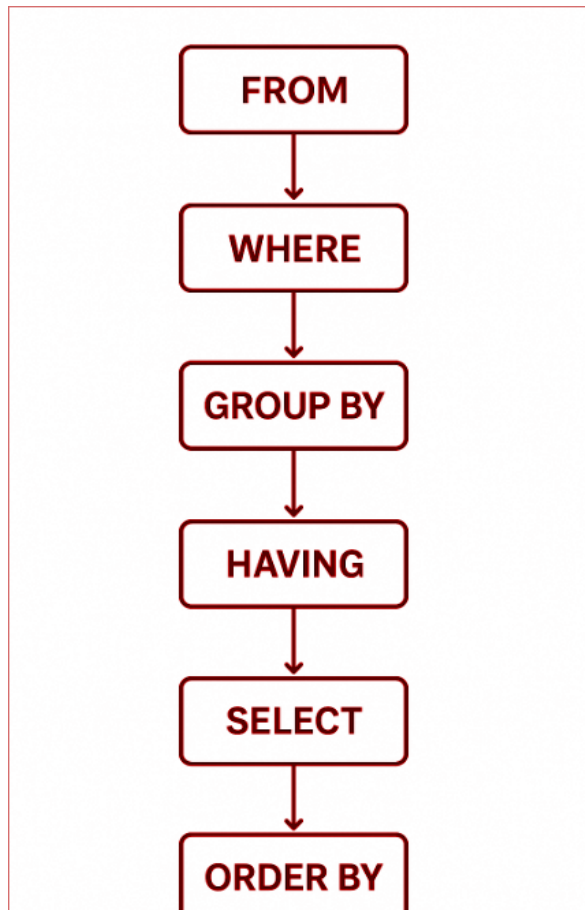
```

SELECT c.customer_id, c.name, o.order_id, o.amount, o.status
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.customer_id, c.name, o.order_id, o.amount, o.status
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

```

Execution Order vs Writing Order

Writing Order	Execution Order
SELECT	FROM
FROM	WHERE
WHERE	GROUP BY
GROUP BY	HAVING
HAVING	SELECT
ORDER BY	ORDER BY



Ex

SELECT cgpa*10 AS marks FROM student WHERE marks > 70; — error

SELECT cgpa*10 AS marks FROM student HAVING marks > 70; -- works

Revision database ;

select amount*10 as selling_price from orders where selling_price > 15000;

ERROR 1054 (42S22): Unknown column 'selling_price' in 'where clause'

MySQL Subqueries

- A subquery is a SELECT statement within another statement.
- All subquery forms and operations that the SQL standard requires are supported, as well as a few features that are MySQL-specific.
- Example:
SELECT * FROM t1 WHERE column1 = (SELECT column1 FROM t2);
In this example, SELECT * FROM t1 ... is the outer query (or outer statement), and (SELECT column1 FROM t2) is the subquery.
- The subquery is nested within the outer query, and can itself contain other subqueries.
- A subquery must always appear within parentheses.

Advantages of Subqueries:

- They allow queries to be structured so that each part can be isolated.
 - They provide alternatives to complex joins and unions.
 - Subqueries are often more readable than complex joins.
-
- A subquery can return a scalar (single value), a single row, a single column, or a table (one or more rows of one or more columns).
 - These are called scalar, column, row, and table subqueries.
 - The outer statement can be SELECT, INSERT, UPDATE, DELETE, SET, or DO.

Ex.

```
DELETE FROM t1
WHERE s11 > ANY
(SELECT COUNT(*) /* no hint */ FROM t2
WHERE NOT EXISTS
(SELECT * FROM t3
WHERE ROW(5*t2.s1,77)=
(SELECT 50,11*s1 FROM t4 UNION SELECT 50,77 FROM
(SELECT * FROM t5) AS t5)));
```

Subquery types :

1. The Subquery as Scalar Operand

A scalar subquery is a subquery that returns exactly one column value from one row. A scalar subquery is a simple operand, and you can use it almost

anywhere a single column value or literal is legal. If the subquery returns 0 rows then the value of scalar subquery expression is NULL and if the subquery returns more than one row then MySQL returns an error.

Example:

```
SELECT * FROM t1 WHERE column1 = (SELECT MAX(column2) FROM t2);
```

2. Comparisons Using Subqueries

A subquery can be used before or after any of the comparison operators. The subquery can return at most one value. The value can be the result of an arithmetic expression or a column function. SQL then compares the value that results from the subquery with the value on the other side of the comparison operator.

non_subquery_operand comparison_operator (subquery)

Operators: =, >, <, >=, <=, <>, !=, <=>

Example:

```
SELECT * FROM t1 WHERE column1 = (SELECT MAX(column2) FROM t2);
```

3. Subqueries with ANY, IN, or SOME

A subquery can be used before or after any of the comparison operators. The subquery can return at most one value. The value can be the result of an arithmetic expression or a column function. SQL then compares the value that results from the subquery with the value on the other side of the comparison operator.

Syntax:

operand comparison_operator ANY (subquery)

operand IN (subquery)

operand comparison_operator SOME (subquery)

ANY - TRUE if the comparison is TRUE for any value returned by subquery.

Example:

```
SELECT s1 FROM t1 WHERE s1 > ANY (SELECT s1 FROM t2);
```

4. Subqueries with ALL

Syntax:

operand comparison_operator ALL (subquery)

ALL ® TRUE if the comparison is TRUE for all returned values.

Example:

SELECT s1 FROM t1 WHERE s1 > ALL (SELECT s1 FROM t2);

5. Row Subqueries

Row subqueries return a single row with multiple columns.

Example:

SELECT * FROM t1 WHERE (col1,col2) = (SELECT col3,col4 FROM t2 WHERE id=10);

6. Subqueries with EXISTS / NOT EXISTS

EXISTS TRUE if subquery returns any rows.

NOT EXISTS TRUE if subquery returns no rows.

Example:

SELECT column1 FROM t1 WHERE EXISTS (SELECT * FROM t2);

Operator	Meaning
IN	Equal to any value in a list
ANY	Compare with any value returned
ALL	Compare with all values returned
EXISTS	True if any row exists in subquery result

7. Correlated Subqueries

A correlated subquery refers to a column from the outer query.

Example:

```
SELECT * FROM t1
```

```
WHERE column1 = ANY (SELECT column1 FROM t2 WHERE t2.column2  
= t1.column2);
```

In this example, the subquery depends on each row of t1.

Correlated subqueries cannot contain LIMIT, OFFSET, or set operations (UNION).

Other rules:

- The correlated column can appear only in the WHERE clause.
- It cannot be in SELECT, JOIN, ORDER BY, GROUP BY, or HAVING.
- It cannot appear inside a derived table in the subquery's FROM list.
- Cannot be inside aggregate functions or window functions.

MySQL Views :

A View is a virtual table that does not store data by itself.

Its contents are derived from one or more base tables using a SELECT query.

Key Characteristics

- A view behaves like a real table with rows and columns but holds no physical data.
- It is built on top of one or more base tables (or even other views).
- Any changes made to the underlying tables are automatically reflected in the view.
- Views are useful for simplifying complex queries and improving data security.

Creating a View

You can create a new view using the **CREATE VIEW** statement along with a **SELECT** query that defines its structure and data source.

Syntax:

```
CREATE [OR REPLACE] VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
[WHERE condition];
```

Example:

```
CREATE VIEW trainer AS
SELECT course_name, trainer
FROM courses;
```

To retrieve data from a view:

```
SELECT * FROM view_name;
```

Modifying a View

To modify or redefine an existing view without dropping it, use the **ALTER VIEW** statement.

Syntax:

```
ALTER VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
[WHERE condition];
```

Advantages of Using Views

1. Data Security:

Views can restrict access to specific columns or rows, showing only the intended data to authorized users.

2. Simplification:
Complex SQL queries can be stored as a view, allowing users to retrieve results with simple SELECT statements.
3. Consistency:
Multiple users can use the same view, ensuring consistent data representation across applications.
4. Abstraction:
Views provide a layer of abstraction over the actual database schema.

Updating Data through Views

- You can perform INSERT, UPDATE, and DELETE operations on a view, and the changes will affect the underlying base tables.
- However, such operations are only allowed if:
 - The view includes all NOT NULL columns from the tables.
 - The view is based on a single table without complex joins or aggregate functions.
- Generally, views are used for querying and reporting, not for data modification.

MySQL Index

An index in MySQL is a data structure (commonly a B-tree or hash table) that improves the speed of data retrieval operations on a table at the cost of additional storage space and slower write operations (INSERT, UPDATE, DELETE).

Think of it like an index in a book — instead of scanning every page to find a topic, MySQL can jump directly to the rows it needs.

How Index Works

When you create an index on a column:

- MySQL stores the sorted values of that column separately.
- When you run a query with a condition like WHERE, JOIN, or ORDER BY, MySQL uses the index to quickly locate matching rows.

Without an index → MySQL does a full table scan (reads all rows).

With an index → MySQL does an index scan (much faster).

Syntax

-- 1. Create index

```
CREATE INDEX index_name ON table_name(column_name);
```

-- 2. Create unique index (no duplicate values)

```
CREATE UNIQUE INDEX index_name ON table_name(column_name);
```

-- 3. Drop (remove) index

```
DROP INDEX index_name ON table_name;
```

-- 4. Show indexes of a table

```
SHOW INDEX FROM table_name;
```

Example

```
CREATE TABLE employees (  
  emp_id INT AUTO_INCREMENT PRIMARY KEY,  
  emp_name VARCHAR(50),  
  department VARCHAR(30),  
  salary DECIMAL(10,2)  
);
```

-- Create an index on department column

```
CREATE INDEX idx_department ON employees(department);
```

-- Query using index

```
SELECT * FROM employees WHERE department = 'IT';
```

MySQL will now use idx_department to quickly find employees in the IT department.

Types of Indexes in MySQL

Type	Description	Example
PRIMARY INDEX	Automatically created for PRIMARY KEY. Unique and not nullable.	PRIMARY KEY (emp_id)
UNIQUE INDEX	Ensures all values in the column are unique.	CREATE UNIQUE INDEX idx_email ON users(email);
INDEX (Non-Unique)	Speeds up searches but allows duplicates.	CREATE INDEX idx_name ON customers(name);
FULLTEXT INDEX	Used for full-text searches in text fields.	CREATE FULLTEXT INDEX idx_post ON articles(content);
SPATIAL INDEX	Used for geometry or spatial data types.	CREATE SPATIAL INDEX idx_location ON maps(geo_point);
COMPOSITE INDEX	Index on multiple columns. Useful for combined searches.	CREATE INDEX idx_name_city ON customers(name, city);

Composite Index Example

```
CREATE INDEX idx_name_city ON customers(name, city);
-- Uses the index
SELECT * FROM customers WHERE name='Ravi' AND city='Pune';
-- Partially uses index (leftmost column rule)
SELECT * FROM customers WHERE name='Ravi';
```

Leftmost Prefix Rule:

If you have an index on (A, B, C) → it works for (A), (A,B), (A,B,C) but not for (B,C) or (C) alone.

Checking Index Usage

```
EXPLAIN SELECT * FROM employees WHERE department = 'IT';
```

This will show the query execution plan, including which index was used.

ACID

- DBMS is the management of data that should remain integrated when any changes are done in it.
- It is because if the integrity of the data is affected, whole data will get disturbed and corrupted.
- Therefore, to maintain the integrity of the data, there are four properties described in the database management system, which are known as the ACID properties.
- The ACID properties are meant for the transaction that goes through a different group of tasks, and there we come to see the role of the ACID properties.

Atomicity

The term atomicity defines that the data remains atomic.

It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all.

It further means that the operation should not break in between or execute partially.

In the case of executing operations on the transaction, the operation should be completely executed and not partially.

User1 1000

user2 2000

Debit 400

credit - 2400

Credit to User 2

Bal - 600

Consistency

- The word consistency means that the value should remain preserved always.
- In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always.
- In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction. The data should always be correct.

User 1 - 600

user2 - 2500

Debit 100

User 3 - 500

Isolation

- The term 'isolation' means separation. In DBMS, Isolation is the property of a database where no data should affect the other one and may occur concurrently.
- In short, the operation on one database should begin when the operation on the first database gets complete. It means if two operations are being performed on two different databases, they may not affect the value of one another.
- In the case of transactions, when two or more transactions occur simultaneously, the consistency should remain maintained.
- Any changes that occur in any particular transaction will not be seen by other transactions until the change is not committed in the memory.

Durability

- Durability ensures the permanency of something. In DBMS, the term durability ensures that the data after the successful execution of the operation becomes permanent in the database.
- The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives.
- However, if gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database.
- For committing the values, the COMMIT command must be used every time we make changes.

Storage Engine

- MySQL table types/storage engines are essential features that can be used effectively for maximizing the database's performance.
- It handles create, read, and update operations for storing and managing the information in a database.
- The following are various table types/storage engines
- supports in MySQL:
 - ISAM
 - MyISAM
 - MERGE
 - InnoDB
 - MEMORY (HEAP)
 - ARCHIVE
 - BDB
 - CSV
 - FEDERATED
- We can use the below query that determines which table types/storage engines our MySQL server supports.

SHOW ENGINES;

show table status;

ISAM

- It is abbreviated as Indexed Sequential Access Method.

- This table type/storage engine has been deprecated and removed from MySQL version 5.x.
- MyISAM now replaces the functionalities of this table.
- The size of an ISAM table is 4 GB, which requires expensive hardware. It is not portable.

MyISAM

- It is an extension of the ISAM storage engine. The MyISAM table types are optimized for compression and speed and can be easily portable between system to system.
- Before version 5.5, if we do not specify the table type during table creation, it was the default storage engine. From version 5.x, InnoDB is used as the default table type/storage engine.
- The MyISAM table size is dependent on the OS and can be up to 256 TB. It can be compressed into read-only tables that save spaces in memory.
- MyISAM table type can store 64 keys per table and contains 1024 bytes maximum key length.
- The MyISAM tables work very fast, but they are not transaction-safe.

Advantages of MyISAM

- If you are new, it will be best to start with MyISAM because it is simple to design and create.
- It is faster than other storage engines in general conditions.
- It provides full-text indexing/searching.

Disadvantages of MyISAM

- MyISAM tables are not transactions-safe.
- It has poor data integrity and crash recovery.
- When we lock the entire table, in that case, MyISAM is slower than InnoDB.

InnoDB Table

- The InnoDB tables in MySQL fully support transaction-safe storage engine with ACID-Compliant.
- It is the first table type that supports foreign keys. The InnoDB tables also provide optimal performance. Its size can be up to 64TB.

- InnoDB tables are also portable between systems to systems similar to MyISAM. InnoDB tables can also be checked and repairs by MySQL whenever necessary.

Advantages of InnoDB

- InnoDB provides optimal performance while processing a large amount of data.
- InnoDB tables arrange our data on the disk based on the primary key.

Disadvantages of InnoDB

- InnoDB tables take more space on disk in comparison with MyISAM.

MySQL Client Programs

Category	Tools	Purpose
Client / Query Tools	mysql, mysqlsh, mysqlpump	Connect to MySQL server and run SQL queries or scripts
Administrative Tools	mysqladmin, mysqldump, mysqlcheck, mysqlimport, mysqlshow	Perform database-level management, backup, repair, etc.
Server Control Tools	mysqld, mysqld_safe, mysqld_multi	Start, stop, or manage MySQL server processes
System Info / Utility Tools	mysqlbinlog, mysqlslap, mysql_upgrade, mysql_secure_installation	Logs, benchmarking, upgrades, and security setup

MySQL: Flow-Control Statements

IF function

- The IF function is one of the parts of the MySQL control flow function, which returns a value based on the given conditions.
- In other words, the IF function is used for validating a function in MySQL.
- It returns values either in a string or numeric form depending upon the context in which this function is used.
- Sometimes, this function is known as IF-ELSE and IF THEN ELSE function.
- The IF function takes three expressions, where the first expression will be evaluated.
- If the first expression evaluates to true, not null, and not zero, it returns the second expression. If the result is false, it returns the third expression.
- Syntax:

IF (expression 1, expression 2, expression 3)

Example 1

SELECT IF(200>350,'YES','NO') as result;

- The IF statement is used in stored programs that implement the basic conditional construct in MySQL. Based on a certain condition, it allows us to execute a set of SQL statements.
- It returns one of the three values True, False, or NULL.
- We can use this statement in three ways IF-THEN, IF-THEN-ELSE, IF-THEN-ELSEIF-ELSE clauses, and can terminate with END-IF.

If-then statement

- This statement executes a set of SQL queries based on certain conditions or expressions. The syntax of the IF- THEN statement is as follows:

IF condition THEN

statements;

END IF;

- In the above syntax, we have to specify a condition for executing the code.

- If the statement evaluates to true, it will execute the statement between IF-THEN and END-IF. Otherwise, it will execute the statement following the END-IF.

EX.

DELIMITER \$\$

```
CREATE PROCEDURE myResult(original_rate float, OUT discount_rate
float)
```

NO SQL

BEGIN

IF (original_rate>200) THEN

SET discount_rate=original_rate*.5;

ELSE

SET discount_rate=original_rate;

END IF;

select discount_rate;

END\$\$

DELIMITER ;

Next, create two variables and set the value for

both as below:

```
mysql> set @p = 150;
```

```
mysql> set @dp = 180;
```

Now, call the stored procedure function to get the

output.

```
mysql> call myResult(@p, @dp)
```

IF-THEN-ELSEIF-ELSE Statement

- If we want to execute a statement based on multiple conditions, this statement can be used. The syntax of the IF-THEN-ELSE statement is given below:

IF condition THEN

statements;

ELSEIF elseif-condition THEN

elseif-statements;

...

...

ELSE

else-statements;

END IF;

Case statements

- The CASE expression validates various conditions and returns the result when the first condition is true.
- Once the condition is met, it stops traversing and gives the output. If it will not find any condition true, it executes the else block.
- When the else block is not found, it returns a NULL value.
- The main goal of MySQL CASE statement is to deal with multiple IF statements in the SELECT clause.

Syntax:

CASE value

WHEN [compare_value] THEN result

[WHEN [compare_value] THEN result ...]

[ELSE result]

END

- The second method is to consider a search_condition in the WHEN clauses, and if it finds, return the result in the corresponding THEN clause.
- Otherwise, it will return the else clause. If else clause is not specified, it will return a NULL value.
- Syntax:

CASE

WHEN [condition] THEN result

[WHEN [condition] THEN result ...]

[ELSE result]

END

Ex. SELECT id, name, marks, class,

CASE class

WHEN 'TY' THEN '3rd Year'

WHEN 'BE' THEN 'Final Year'

WHEN 'SY' THEN '2nd Year'

ELSE 'NONE'

END AS year from student;

Loop

- Similar to other programming languages MySQL provides support for the flow control statements such as IF, CASE, ITERATE, LEAVE LOOP, WHILE, and REPEAT.
- You can use these statements in the stored programs (procedures), and RETURN in stored functions. You can use one Flow Control Statement with in another.
- The LOOP is a compound MySQL statement which is used to execute a single or set of statements repeatedly.
- syntax

begin_label: LOOP

statement_list

END LOOP end_label

- Where, `statement_list` is a single or set of statements that are to be repeated. `begin_label` and `end_label` are the optional labels of the LOOP statement.
- The statement(s) in the LOOP are executed repeatedly till the loop is terminated. You can terminate the LOOP using the LEAVE statement.
- When used in a function the LOOP can also be terminated using the RETURN statement. Each statement in the LOOP ends with a semi colon (or. the current delimiter).

Delimiter ;

While Loop

- The WHILE is a compound MySQL statement which is used to execute a single or set of statements repeatedly as long as the specified condition is TRUE.
- Syntax:

`begin_label: WHILE search_condition DO`

`statement_list`

`END WHILE end_label`

- Where, `statement_list` is a single or set of statements that are to be repeated. `begin_label` and `end_label` are the optional labels of the WHILE statement.

Repeat Loop

- The REPEAT statement in MySQL is used to repeat the given set of statements (or statement) until the value of the given search condition is TRUE.
- The statement(s) in the LOOP ends with a semi colon (or. the current delimiter).
- Syntax:

`begin_label: REPEAT`

`statement_list`

`UNTIL search_condition`

`END REPEAT end_label`

Leave Statement

- The LEAVE statement in MySQL is used to exit the LOOP, REPEAT, WHILE statements or, BEGIN...END statements.
- It cannot be used outside these statements.
- Syntax:

LEAVE label

- Where, label is the label of the LOOP or, REPEAT or, WHILE statement.

Iterate Statement

- The ITERATE statement is used to restart the LOOP, REPEAT or, WHILE statements.
- It cannot be used outside these statements.
- Syntax:

ITERATE label

- Where, label is the label of the LOOP or, REPEAT or, WHILE statement.

MySQL: Flow-Control Statements

IF function

- The IF function is one of the parts of the MySQL control flow function, which returns a value based on the given conditions.
- In other words, the IF function is used for validating a function in MySQL.
- It returns values either in a string or numeric form depending upon the context in which this function is used.
- Sometimes, this function is known as IF-ELSE and IF THEN ELSE function.
- The IF function takes three expressions, where the first expression will be evaluated.
- If the first expression evaluates to true, not null, and not zero, it returns the second expression. If the result is false, it returns the third expression.
- Syntax:

IF (expression 1, expression 2, expression 3)

Example 1

```
SELECT IF(200>350, 'YES', 'NO') as result;
```

- The IF statement is used in stored programs that implement the basic conditional construct in MySQL. Based on a certain condition, it allows us to execute a set of SQL statements.
- It returns one of the three values True, False, or NULL.
- We can use this statement in three ways IF-THEN, IF-THEN-ELSE, IF-THEN-ELSEIF-ELSE clauses, and can terminate with END-IF.

If-then statement

- This statement executes a set of SQL queries based on certain conditions or expressions. The syntax of the IF- THEN statement is as follows:

```
IF condition THEN
```

```
statements;
```

```
END IF;
```

- In the above syntax, we have to specify a condition for
- executing the code.
- If the statement evaluates to true, it will execute the statement between IF-THEN and END-IF. Otherwise, it will execute the statement following the END-IF.

EX. DELIMITER \$\$

```
CREATE PROCEDURE myResult(original_rate float, OUT discount_rate
float)
```

```
NO SQL
```

```
BEGIN
```

```
IF (original_rate>200) THEN
```

```
SET discount_rate=original_rate*.5;
```

```
ELSE
```

```
SET discount_rate=original_rate;
```

```
END IF;
```

```
select discount_rate;
```

```
END$$
```

DELIMITER ;

Next, create two variables and set the value for both as below:

```
mysql> set @p = 150;
```

```
mysql> set @dp = 180;
```

Now, call the stored procedure function to get the output.

```
mysql> call myResult(@p, @dp)
```

IF-THEN-ELSEIF-ELSE Statement

- If we want to execute a statement based on multiple conditions, this statement can be used. The syntax of the IF-THEN-ELSE statement is given below:

IF condition THEN

statements;

ELSEIF elseif-condition THEN

elseif-statements;

...

...

ELSE

else-statements;

END IF;

Case statements

- The CASE expression validates various conditions and returns the result when the first condition is true.

- Once the condition is met, it stops traversing and gives the output. If it will not find any condition true, it executes the else block.
- When the else block is not found, it returns a NULL value.
- The main goal of MySQL CASE statement is to deal with multiple IF statements in the SELECT clause.

Syntax:

CASE value

WHEN [compare_value] THEN result

[WHEN [compare_value] THEN result ...]

[ELSE result]

END

- The second method is to consider a search_condition in the WHEN clauses, and if it finds, return the result in the corresponding THEN clause.
- Otherwise, it will return the else clause. If else clause is not specified, it will return a NULL value.
- Syntax:

CASE

WHEN [condition] THEN result

[WHEN [condition] THEN result ...]

[ELSE result]

END

Ex. SELECT id, name, marks, class,

CASE class

WHEN 'TY' THEN '3rd Year'

WHEN 'BE' THEN 'Final Year'

WHEN 'SY' THEN '2nd Year'

ELSE 'NONE'

END AS year from student;

Loop

- Similar to other programming languages MySQL provides support for the flow control statements such as IF, CASE, ITERATE, LEAVE LOOP, WHILE, and REPEAT.
- You can use these statements in the stored programs (procedures), and RETURN in stored functions. You can use one Flow Control Statement within another.
- The LOOP is a compound MySQL statement which is used to execute a single or set of statements repeatedly.
- syntax

```
begin_label: LOOP
```

```
statement_list
```

```
END LOOP end_label
```

- Where, statement_list is a single or set of statements that are to be repeated. begin_label and end_label are the optional labels of the LOOP statement.
- The statement(s) in the LOOP are executed repeatedly till the loop is terminated. You can terminate the LOOP using the LEAVE statement.
- When used in a function the LOOP can also be terminated using the RETURN statement. Each statement in the LOOP ends with a semi colon (or. the current delimiter).

While Loop

- The WHILE is a compound MySQL statement which is used to execute a single or set of statements repeatedly as long as the specified condition is TRUE.
- Syntax:

```
begin_label: WHILE search_condition DO
```

```
statement_list
```

```
END WHILE end_label
```

- Where, `statement_list` is a single or set of statements that are to be repeated. `begin_label` and `end_label` are the optional labels of the `WHILE` statement.

Repeat Loop

- The `REPEAT` statement in MySQL is used to repeat the given set of statements (or statement) until the value of the given search condition is `TRUE`.
- The statement(s) in the `LOOP` ends with a semi colon (or. the current delimiter).
- Syntax:

`begin_label: REPEAT`

`statement_list`

`UNTIL search_condition`

`END REPEAT end_label`

Leave Statement

- The `LEAVE` statement in MySQL is used to exit the `LOOP`, `REPEAT`, `WHILE` statements or, `BEGIN...END` statements.
- It cannot be used outside these statements.
- Syntax:

`LEAVE label`

- Where, `label` is the label of the `LOOP` or, `REPEAT` or, `WHILE` statement.

Iterate Statement

- The `ITERATE` statement is used to restart the `LOOP`, `REPEAT` or, `WHILE` statements.
- It cannot be used outside these statements.
- Syntax:

`ITERATE label`

- Where, `label` is the label of the `LOOP` or, `REPEAT` or, `WHILE` statement.

MySQL Function

- A stored function in MySQL is a set of SQL statements that perform some task/operation and return a single value.
- It is one of the types of stored programs in MySQL. When you will create a stored function, make sure that you have a CREATE ROUTINE database privilege.
- Generally, we used this function to encapsulate the common business rules or formulas reusable in stored programs or SQL statements.
- The stored function is almost similar to the procedure in MySQL, but it has some differences that are as follows:
 - The function parameter may contain only the IN parameter but can't allow specifying this parameter, while the procedure can allow IN, OUT, INOUT parameters.
 - The stored function can return only a single value defined in the function header.
 - The stored function may also be called within SQL statements.
 - It may not produce a result set.
- Syntax

DELIMITER \$\$

CREATE FUNCTION fun_name(fun_parameter(s))

RETURNS datatype

[NOT] {Characteristics}

fun_body;

- Fun_name
 - It is the name of the stored function that we want to create in a database. It should not be the same as the built-in function name of MySQL.
- Fun_parameter
 - It contains the list of parameters used by the function body. It does not allow to specify IN, OUT, INOUT parameters.
- Datatype
 - It is a data type of return value of the function. It should any valid MySQL data type.
- Characteristics
 - The CREATE FUNCTION statement only accepted when the characteristics (DETERMINISTIC, NO SQL, or READS SQL DATA) are defined in the declaration.
- Fun_body

- This parameter has a set of SQL statements to perform the operations. It requires at least one RETURN statement.
- When the return statement is executed, the function will be terminated automatically. The function body is given below: BEGIN -- SQL statements END \$\$ DELIMITER

```
DELIMITER //
```

```
CREATE FUNCTION function_name(parameter_list)
```

```
RETURNS data_type
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    -- local variables (optional)
```

```
    -- logic / computation
```

```
    RETURN value;
```

```
END //
```

```
DELIMITER ;
```

MySQL Procedure

- A **procedure** in MySQL is a type of **stored program** that is stored in the database and can be executed (called) when needed.
- It contains a set of SQL statements (and optionally control structures like IF, LOOP, WHILE) that are grouped together under a single name.
- Think of it as a **function without a return value** – it performs a task, but instead of returning a single value, it may process multiple queries, modify data, or return result sets.

Why use Procedures?

- **Reusability** – Write once, use many times.
- **Modularity** – Break complex logic into smaller procedures.
- **Security** – Users can be given permission to execute a procedure without directly accessing the tables.
- **Performance** – Reduces network traffic since logic is executed on the server side.

Syntax

DELIMITER \$\$

CREATE PROCEDURE procedure_name(IN parameter1 datatype, OUT parameter2 datatype, INOUT parameter3 datatype)

BEGIN

-- SQL statements

END \$\$

DELIMITER ;

Parameters

IN parameters

IN is the default mode. When defining an IN parameter in a stored procedure, the calling program must pass an argument to the stored procedure.

Additionally, the value of an IN parameter is protected. This means that even if you change the value of the IN parameter inside the stored procedure, its original value remains unchanged after the stored procedure ends. In other words, the stored procedure works only on the copy of the IN parameter.

OUT parameters

The value of an OUT parameter can be modified within the stored procedure, and its updated value is then passed back to the calling program.

Note that stored procedures cannot access the initial value of the OUT parameter when they begin.

INOUT parameters

An INOUT parameter is a combination of IN and OUT parameters. This means that the calling program may pass the argument, and the stored procedure can modify the INOUT parameter and pass the new value back to the calling program.

Cursor

- A cursor in a database is a construct which allows you to iterate/traverse the records of a table.
- In MySQL you can use cursors within a stored program such as procedures or functions.
- In other words, you can iterate through the records of a table from a MySQL stored program using the cursors.

Cursor Types in MySQL

- MySQL provides embedded cursors:
 - READ ONLY – Using these cursors you cannot update any table.
 - Non-Scrollable – You can retrieve records in only one direction (top to bottom).
 - Asensitive – These cursors are insensitive to changes made in the table after cursor creation.
 - If new records are added to the table, they won't appear in the already created cursor.

Steps to Use a Cursor

When declaring cursors in a stored program:

- Cursor declarations must follow variable and condition declarations.

Steps to use a cursor (in order):

1. DECLARE the cursor using the DECLARE statement.
2. DECLARE variables and conditions.
3. OPEN the cursor using the OPEN statement.
4. FETCH records from the table using the FETCH statement.
5. CLOSE the cursor using the CLOSE statement.

DECLARE CURSOR

- The stored function is almost similar to the procedure in MySQL but with key differences:
 - Functions can have only IN parameters (no OUT or INOUT).
 - Functions return only one value.
 - Functions can be called within SQL statements.
 - Functions may not produce a result set.

DECLARE CURSOR Syntax

- You can declare a cursor and associate it with a SELECT statement (without INTO clause).

DECLARE cursor_name CURSOR FOR select_statement;

- Cursor declaration must precede handler declarations.
 - You can use cursors inside a single stored program.
-

OPEN CURSOR

Syntax:

OPEN cursor_name;

Parameter:

- cursor_name: name of the already declared cursor.
-

FETCH CURSOR

- After opening, use FETCH to retrieve each row.

Syntax:

FETCH [NEXT [FROM]] cursor_name INTO variable_list;

Parameters:

- `cursor_name` – name of the cursor.
- `variable_list` – comma-separated variables to store fetched values.

CLOSE CURSOR

Syntax:

`CLOSE cursor_name;`

Parameter:

- `cursor_name`: name of the cursor.

It is a good practice to always close a cursor when it is no longer used.

When working with MySQL cursor, you must also declare a NOT FOUND handler to manage the situation when the cursor cannot find any row.

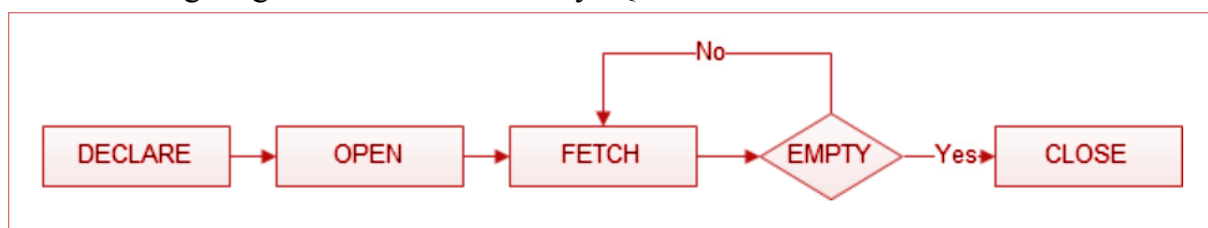
Each time you call the `FETCH` statement, the cursor attempts to read the next row in the result set. When the cursor reaches the end of the result set, it will not be able to retrieve the data, and a condition is raised. The handler is used to handle this condition.

To declare a NOT FOUND handler, you use the following syntax:

`DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;`

The `finished` is a variable to indicate that the cursor has reached the end of the result set.

The following diagram illustrates how MySQL cursor works.



Trigger

- A trigger in MySQL is a set of SQL statements that reside in the system catalog.
- It is a special type of stored procedure that is invoked automatically in response to an event.
- Each trigger is associated with a table, and is activated on any DML statement such as INSERT, UPDATE, or DELETE.
- A trigger is a special procedure because it cannot be called directly like a stored procedure.
- Main difference:
 - Trigger is called automatically when a data modification happens.
 - Stored procedure must be called explicitly.

Types of Triggers

According to the SQL standard, there are two types:

1. Row-Level Trigger

- Activated for each row affected by a triggering statement.
- Example: If multiple rows are inserted or updated, the row trigger fires once for each row.

2. Statement-Level Trigger

- Fired once for each event on a table (regardless of the number of rows affected).

Why Use Triggers

- Enforce business rules.
- Maintain audit trails and logs.
- Ensure data integrity.
- Execute scheduled tasks.
- Improve performance (no recompilation needed).
- Reduce client-side logic.
- Easier to maintain.

Limitations of Triggers

- MySQL triggers only allow extended validations, not all constraints.
 - Example: For simple checks use NOT NULL, UNIQUE, CHECK, FOREIGN KEY.
- They execute invisibly to the client, making debugging harder.
- They can increase server overhead.

Trigger Events

MySQL allows six trigger actions:

1. BEFORE INSERT – before insertion
2. AFTER INSERT – after insertion
3. BEFORE UPDATE – before update
4. AFTER UPDATE – after update
5. BEFORE DELETE – before deletion
6. AFTER DELETE – after deletion

Naming Conventions

- Always use unique names for triggers.
- Good practice: same naming pattern for similar triggers across tables.

Naming format:

(BEFORE | AFTER) table_name (INSERT | UPDATE | DELETE)

Creating a Trigger

Syntax:

```
CREATE TRIGGER trigger_name
  (AFTER | BEFORE) (INSERT | UPDATE | DELETE)
  ON table_name FOR EACH ROW
BEGIN
  -- variable declarations
  -- trigger code
END;
```

Trigger Components

- `trigger_name`: Unique name for the trigger.
- `trigger_time`: Defines when the trigger runs — BEFORE or AFTER.
- `trigger_event`: Defines the event — INSERT, UPDATE, or DELETE.
- `table_name`: The table associated with the trigger.

Note: A trigger can be defined for only one event at a time.

Accessing OLD and NEW Values

- The trigger body can access columns affected by DML operations using:
 - `OLD.col_name` → Value before the update or delete.
 - `NEW.col_name` → Value after the insert or update.

Example:

`OLD.message_info`

`NEW.message_info`

`Insert into( )`

```
Student ( rollNo, Name, Marks, status )  
insert(1, vipul, 101);
```

```
Create trigger Student_trigger1
```

```
Before insert
```

```
On Student
```

```
For each row
```

```
Begin
```

```
    If marks <100 & marks > 0
```

Then
Insert Studnet

End;

student_Logs(rollNo, DATE & TIME)

Create trigger Student_trigger2

After insert

On Student

For each row

Begin

Insert into student_logs values (rollNo, Now());

End;

Update studnet set marks=90 where marks=101;

Create trigger Student_trigger3

After update

On Student

For each row

Begin

Insert the old value, msg into logs table.

End;

Insert into student(name, marks) value ("abc", 75);

Create trigger a before insert on student for each row

Begin

 If new.marks >=70 then set status = “pass”;

end;

Student

id	name	marks	status
1	Vipul	100	pass
1	navin	100	pass
2	Anushka, Ganesh	90	pass

id	marks	status
1		
2		

1	vipul
1	navin
2	Anushka
2	Ganesh